Lab 13 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — `host not found in upstream "api"`

Answer. In lab 07, the Podman network's DNS gave a name to every **container**. In the cluster, a Pod has **no** DNS name: only **Services** do. nginx resolves `api` at startup (lab 07), finds nothing, and exits with an error; the kubelet relaunches it, hence `CrashLoopBackOff`. You must create the `api` Service (selector `app: api`, port 8080): CoreDNS then creates the entry, and at the next backoff relaunch nginx starts — `web` recovers on its own because `restartPolicy: Always` keeps retrying indefinitely.

Why. Naming is the Service's responsibility, not the Pod's: Pods are too ephemeral to deserve a name.

Nuance. In a complete manifest, you apply the Service **before** or **with** the front end's Deployment; the order within one `apply -f folder/` is alphabetical, hence the habit of putting everything in one file or letting the backoff do its job.

Example.

```
kubectl apply -f service-api.yaml && kubectl rollout status deployment/web --timeout=120s   # successfully rolled out
```

Question 2 — Cluster IP, DNS, EndpointSlices

Answer. The **cluster IP** (`10.43.x.x`) is a virtual address assigned by the API server when the Service is created, stable as long as it exists. The **DNS name** `api.lab13.svc.cluster.local` is maintained by CoreDNS, which watches Services. The **EndpointSlices** are the list of `IP:port` of the Pods that carry the selector's label **and** are ready, maintained by the endpoints controller. `ping` fails because the IP exists on no interface: kube-proxy's rules only translate TCP/UDP connections to a Service port, not ICMP.

Why. The Service is a distributed NAT abstraction, not a machine.

Nuance. With kube-proxy in IPVS mode (or Cilium without kube-proxy), the IP may answer `ping` — do not rely on `ping` to diagnose a Service.

Example.

```
kubectl get svc api -o jsonpath='{.spec.clusterIP}' ; kubectl get endpointslices -l kubernetes.io/service-name=api
```

Question 3 — Endpoints follow the Pods

Answer. After scaling to 3 replicas: **three** addresses (as soon as the third Pod is ready). After deleting a Pod: its address disappears **immediately**, then the replacement Pod's address appears once it is ready (a few seconds with two addresses). After breaking the readiness: that Pod's address is **removed**, two remain, the Pod keeps running. The client calling `api:8080` noticed nothing, except a dropped connection if it was precisely on the deleted Pod at the wrong moment.

Why. The endpoints controller reacts to Pod changes (creation, deletion, `Ready` condition) and kube-proxy updates its rules in under a second.

Nuance. Between the moment a Pod receives `SIGTERM` and its removal from the endpoints, a few requests may still reach it: that is why an application must handle graceful shutdown (lab 03) and why a `preStop` of a few seconds is sometimes added.

Example.

```
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
```

Question 4 — A Service with no endpoints

Answer. Either the **selector** matches no Pod label (`app: apis`, `app: backend`, a typo), or the Pods are **not ready** — but here they are `1/1`, so it is the selector. `kubectl get pods --show-labels` compared with `kubectl get svc api -o jsonpath='{.spec.selector}'` settles it. Fix: correct the selector (or the template's labels) and reapply; for the readiness case, fix the probe.

Why. The EndpointSlice is computed from the selector; no match, no address, and Traefik, which routes to endpoints, has nothing to route to: `503`.

Nuance. `kubectl describe svc api` shows `Endpoints: <none>` in plain text; that is the line to look for.

Example.

```
kubectl get svc api -o jsonpath='{.spec.selector}' ; echo ; kubectl get pods --show-labels |
head -3
```

Question 5 — Three ways to expose the front end

Answer. ClusterIP: no URL from the host, only `kubectl port-forward svc/web 18080:80` then `http://localhost:18080`. NodePort: `http://<node-ip>:30080` (and often `http://localhost:30080` from WSL). LoadBalancer: `http://<EXTERNAL-IP>:8081`, where `EXTERNAL-IP` is the node's IP because k3s's **ServiceLB** launches a `svclb-web-lb` Pod that opens the port on the node — bare Kubernetes would leave `<pending>` for lack of a provider. A second LoadBalancer on port 80 would stay `<pending>` because ServiceLB cannot open the same port twice on the same node, and Traefik already holds it.

Why. ServiceLB is a "poor man's" load balancer: one host port per Service, on every node.

Nuance. In the cloud, the same YAML orders a real load balancer with its own IP; ports no longer compete, but each one costs.

Example.

```
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name   #
pod/svclb-web-lb-…
```

Question 6 — The short name only works inside the namespace

Answer. A Pod's `/etc/resolv.conf` contains `search lab13.svc.cluster.local svc.cluster.local cluster.local` and `nameserver 10.43.0.10` (CoreDNS). From `lab13`, `api` becomes `api.lab13.svc.cluster.local` through the first suffix. From `default`, the first suffix is `default.svc.cluster.local`: `api.default.svc.cluster.local` does not exist, nor does `api.svc.cluster.local`. You must write `api.lab13` (the second suffix completes it to `api.lab13.svc.cluster.local`) or the full name.

Why. The `search` line is generated by the kubelet with the Pod's namespace; `ndots:5` makes it try the suffixes before a direct lookup.

Nuance. Calling a Service in another namespace is allowed by default; a NetworkPolicy (lab 16) can forbid it.

Example.

```
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl run -n default -it --rm t --image=docker.io/library/alpine -- wget -qO-
http://api.lab13:8080/actuator/health
```

Question 7 — Connections, not requests

Answer. kube-proxy picks a Pod **when each TCP connection is opened** and never looks at the content. An HTTP/1.1 client with *keep-alive* (or a JDBC pool) opens few connections and keeps them: all its requests go to the **same** Pod, and three replicas only share the load if there are enough clients. The Ingress (Traefik), on the other hand, terminates the client's connection and opens its own towards the Pods: it distributes **per HTTP request**, and can even do sticky or weighted routing.

Why. The Service is a layer-4 mechanism; layer-7 load balancing needs a proxy that understands HTTP.

Nuance. For a database (long connections), this is acceptable and even desirable; for an API called by a single front end, it is the argument for going through the Ingress rather than through the nginx proxy in the `web` Pod.

Example.

```
for i in 1 2 3 4 ; do kubectl exec deploy/web -- wget -qO- http://api:8080/ ; done   # each
wget = new connection = random Pod
```

Question 8 — `port` and `targetPort`

Answer. `port: 80` is the **Service's** port (the one you call: `api:80`, i.e. `http://api/`); `targetPort: 8080` is the **container's** port that kube-proxy translates to. Nobody serves `api:8080`, hence `Connection refused`. nginx must write `proxy_pass http://api/;` (or `http://api:80/`).

Why. The Service decouples the exposed port from the real one, which lets you change either without the other — and give every Service a "clean" port.

Nuance. `targetPort` can be a container port **name** (`targetPort: http` if the container declares `name: http`), which survives a number change in the image.

Example.

```
kubectl get svc api -o jsonpath='{.spec.ports[0].port} -> {.spec.ports[0].targetPort}' ; echo
```

Question 9 — The path of a request

Answer. Windows browser → WSL forwarding → node port **80**, opened by the `svclb-traefik` Pod (ServiceLB) → Service `traefik` → **Traefik** Pod, which reads the `web` Ingress (`/` → Service `web`) → Service `web` → `web` Pod (**nginx**), whose configuration relays `/api/` → Service `api` → an `api` Pod, which answers `{"status":"UP"}`. Two reverse proxies: Traefik then nginx. Simplification: add an Ingress rule `path: /api` → Service `api` port 8080 (with a prefix rewrite), and remove the `location /api/` from nginx — Traefik routes directly.

Why. Each component only knows the next one: that is what makes every link replaceable.

Nuance. The `/api/x` → `/x` rewrite is done with a *middleware* (`stripPrefix`) in Traefik, with a `rewrite-target` annotation in ingress-nginx: the syntax depends on the controller, not on the standard Ingress object.

Example.

```
kubectl describe ingress web | grep -A4 Rules
```

Question 10 — Ingress and Ingress controller

Answer. The `Ingress` is an API object: routing rules, nothing more. The *Ingress controller* is a **program** (Traefik, nginx…) deployed in the cluster, which watches those objects and configures its reverse proxy accordingly. Without a controller, an Ingress is accepted, stored, and **has no effect**: no `ADDRESS`, no traffic. A `503` from Traefik means Traefik did receive the request, found the rule, but the targeted Service has **no ready endpoint**: the problem is downstream (Pods, readiness, selector), Traefik is only the messenger.

Why. Kubernetes defines the interface, not the implementation; same model as CNI (network) and CSI (storage).

Nuance. A cluster can have several controllers; `spec.ingressClassName` (or the default class, `traefik` with k3s) says which one takes the object.

Example.

```
kubectl get ingressclass ; kubectl get ingress web -o jsonpath='{.spec.ingressClassName}'
```

Question 11 — The Ingress with no `ADDRESS` on EKS

Answer. An **Ingress controller** is missing: k3s ships one (Traefik), EKS does not. The Ingress objects are stored but nobody reads them; `ADDRESS` stays empty. A controller must be installed (ingress-nginx, Traefik, or the AWS Load Balancer Controller) and, depending on the case, `ingressClassName` set. It "worked" on k3s because Traefik was already there, with its default class.

Why. Distributions choose what they bundle; a managed cluster leaves that choice to the platform team.

Nuance. On EKS, the `LoadBalancer` Service works with nothing installed because the cloud integration **is** provided (the cloud-controller-manager): the opposite of k3s, where ServiceLB simulates it.

Example.

```
kubectl get pods -A | grep -i -E 'ingress|traefik'   # empty = no controller
```

Question 12 — `/api/` through nginx or through the Ingress

Answer. Same origin = no CORS, a single URL to configure, the front end does not know where the API lives (lab 07). In our stack, the `/api/` routing can be done by **nginx** (the `location /api/` of the `web` Pod) or by the **Ingress** (a `path: /api` rule). Entrusting it to the Ingress: one proxy instead of two, the `web` image no longer needs to know the `api` name (hence no more crash at startup), Traefik's per-request balancing across the `api` Pods, TLS and headers in one place, and a front end reduced to static files.

Why. Routing is a platform responsibility; the static front end does not have to carry it.

Nuance. The proxy inside nginx remains useful on the workstation (Compose, lab 09) where there is no Ingress: some teams keep both configurations.

Example.

```
- path: /api
  pathType: Prefix
  backend: { service: { name: api, port: { number: 8080 } } }
```

Question 13 — Reaching the database from Windows

Answer. Increasing exposure: `port-forward` (a tunnel encrypted through the API server, valid for the duration of the command, visible to you alone) < NodePort (a port open on every node, for the whole network) < LoadBalancer (a dedicated IP, public in the cloud). In production, only the first is acceptable, and even then for a one-off diagnosis with the proper RBAC rights (lab 16): a database never has a port to the outside. What is still missing: a `db` **Service** (ClusterIP, or *headless* for a StatefulSet, lab 15) so that `db` has a name in the cluster.

Why. Each Service type widens the exposure surface; a human's need is not the application's.

Nuance. In the workplace, human access to databases goes through a bastion or an audit tool (Teleport, Boundary), never through a node port.

Example.

```
kubectl port-forward svc/db 5432:5432   # then DBeaver on localhost:5432, for the duration of
the session
```